TÉCNICO
UNIVERSIDADE
DE LISBOA

Digital Systems Design

MSc in Electrical and Computer Engineering

Lab. Proj. 2 – Fully-Connected Layer with ReLU

Authors:

Rodrigo Caldeira (106963) rodrigo.caldeira@tecnico.ulisboa.pt
Guilherme Garcia (105917) guilherme.f.garcia@tecnico.ulisboa.pt
Diogo Diniz (106196) diogo.cruz.diniz@tecnico.ulisboa.pt

Group 7

2026/2027 – 1st Semester

Contents

1 Introduction **2**

2 System architecture **2**
2.1 Data flow graph . . . 2
2.2 Priority list . . . 2
2.3 Scheduling . . . 3
2.3.1 Chosen implementation . . . 3
2.3.2 Sequential implementation . . . 4
2.3.3 Final implementation . . . 4

3 Circuit design **5**
3.1 Overall architecture . . . 5
3.2 Control Unit . . . 5
3.2.1 Inputs and outputs . . . 5
3.2.2 State machine . . . 5
3.3 Datapath . . . 6
3.4 Testbench . . . 6

4 Synthesis results **7**

5 Implementation results **7**
5.1 Critical path . . . 8

6 Pipelining **8**
6.1 Current architecture pipelining . . . 8
6.2 Maximum Throughput . . . 9

7 Conclusion **9**

1 Introduction

This report presents the design and simulation of a fully-connected layer with ReLU. The system was designed in VHDL and tested on a simulated AMD Artix-7 FPGA.

The system is composed of three main blocks: Datapath, Control Unit and Test Bench. Behavioral, Post Synthesis and Post Implementation simulations verified a variety of example cases, testing all the identified edge cases. The physical implementation achieved the following major results:

- Low resource usage: 266 Slice LUTs (1.28%), 88 registers (0.21%) and no DSP48E1 slices (multipliers implemented with LUTs).
- Successful timing closure at 100 MHz with a Worst Negative Slack (WNS) of 0.512 ns which implies a maximum achievable clock frequency of $f_{\max} = 105.4$ MHz.

2 System architecture

2.1 Data flow graph

The dataflow graph for one iteration of the inner loop can be seen in Figure 1.

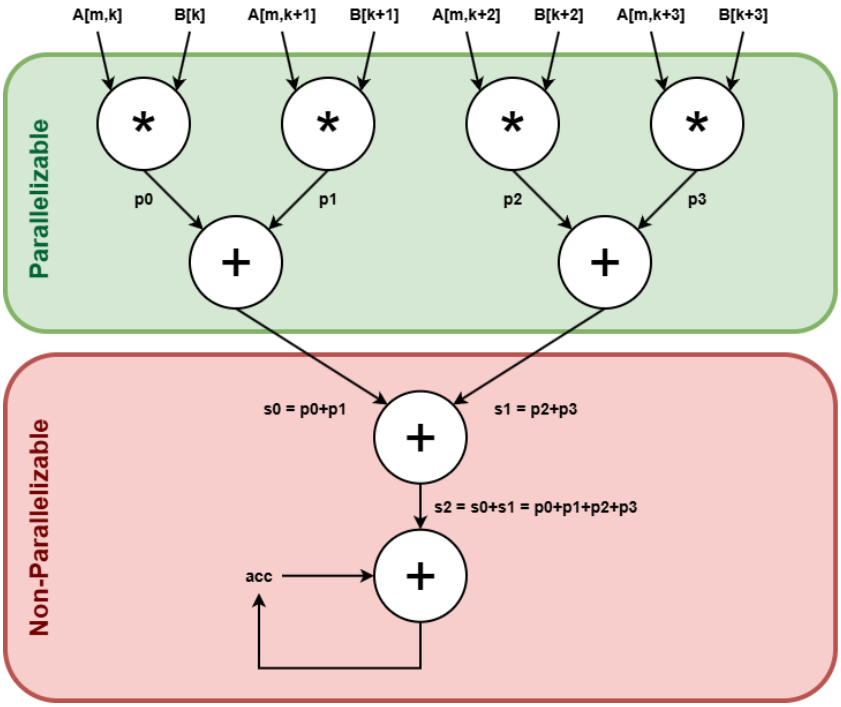


Figure 1: Dataflow graph of the inner loop, showing parallelizable operations.

There is a clear dependency between iterations since the accumulator is shared between them. The register can be used by only one iteration at a time, and all iterations must add their multiplication results to this register to get the final output.

2.2 Priority list

Defining priorities based on the critical path results in the following priority list:

1. p0
2. p1
3. p2
4. p3
5. s0
6. s1
7. s2
8. s2+acc

Figure 2 takes the dataflow graph and adds priorities in order to make the critical path metric more obvious.

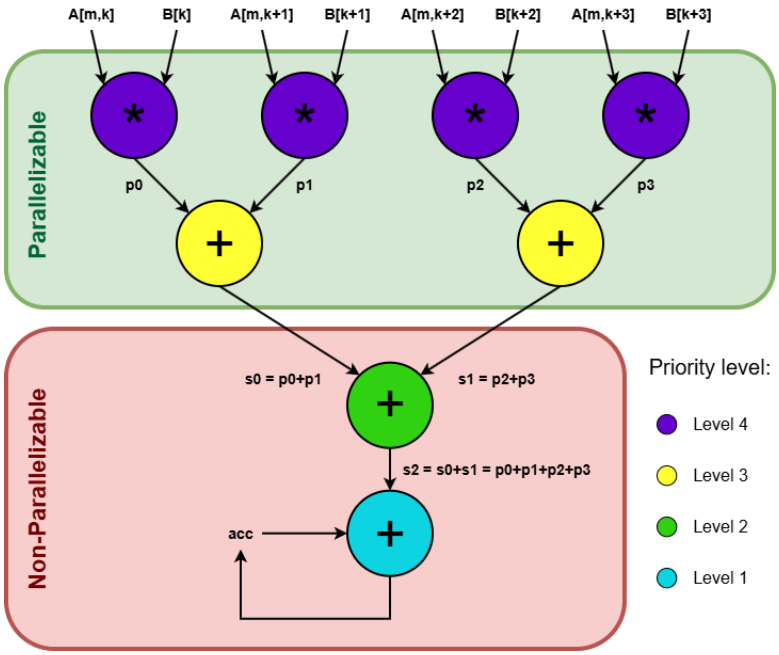


Figure 2: Dataflow graph of the inner loop with priorities assigned to each operation according to the critical path.

2.3 Scheduling

Based on the priority list, the focus on performance, and the hardware constraints, scheduling was attempted with different approaches as seen in Figure 3.

2.3.1 Chosen implementation

Since the objective of this work is to maximize performance, the design with two multipliers and the triple-input adder, which minimizes latency, was the obvious choice. A valid concern with this approach is the potential increase in clock period due to a larger adder, which would negate the gain in number of cycles. This is not the case since the multiplication latency is the limiting factor. This fact will be verified through simulation in section 5.1.

The number of cycles required for this implementation is:

1. The multiply-accumulate of 4 values takes 3 cycles.
2. There are 64 multiply-accumulate operations for each output. With 4 values accumulated per iteration, there need to be $64/4 = 16$ iterations of the inner loop.
3. With 16 outputs, and considering the extra cycle needed to write the output, this turns into a total of: $(3 \times 16 + 1) \times 16 = 784$ cycles.

2.3.2 Sequential implementation

A sequential implementation with 1 multiplier and 1 adder is quite limited. Without accounting for the activation of each output (0 cycles), the number of cycles required for this implementation is:

1. Each multiply-accumulate operation takes 2 cycles.
2. Each output requires 64 multiply-accumulate operations.
3. With 16 outputs, and considering the extra cycle needed to write the output, this turns into a total of: $(2 \times 64 + 1) \times 16 = 2064$ cycles.

2.3.3 Final implementation

The implementation described in Section 2.3.1 was the one originally presented in class. As was observed, it failed in the Post-synthesis timing simulation. We tested other strategies to correct the issue and evaluated performance and concluded that extra states and clock cycles were needed to achieve correctness.

In our final implementation, two 'wait states' were introduced in the inner loop. These feed the address to be read to the input and weight BRAMs and allow one clock cycle for them to be processed before attempting to read their values. This meant that each iteration of the inner loop was now 5 cycles instead of 3, which was the same value obtained in the first scheduling approach. However, after applying the same correction to this scheduling we observed that it too suffered with the addition of new cycles, now taking 7 cycles, so our triple-input adder solution still stands as the fastest from the scheduling we initially studied.

With the new scheduling, the total number of cycles changed:

1. The multiply-accumulate of 4 values takes 5 cycles.
2. There are 64 multiply-accumulate operations for each output. With 4 values accumulated per iteration, there need to be $64/4 = 16$ iterations of the inner loop.
3. With 16 outputs, and considering the extra cycle needed to write the output, this turns into a total of: $(5 \times 16 + 1) \times 16 = 1296$ cycles.

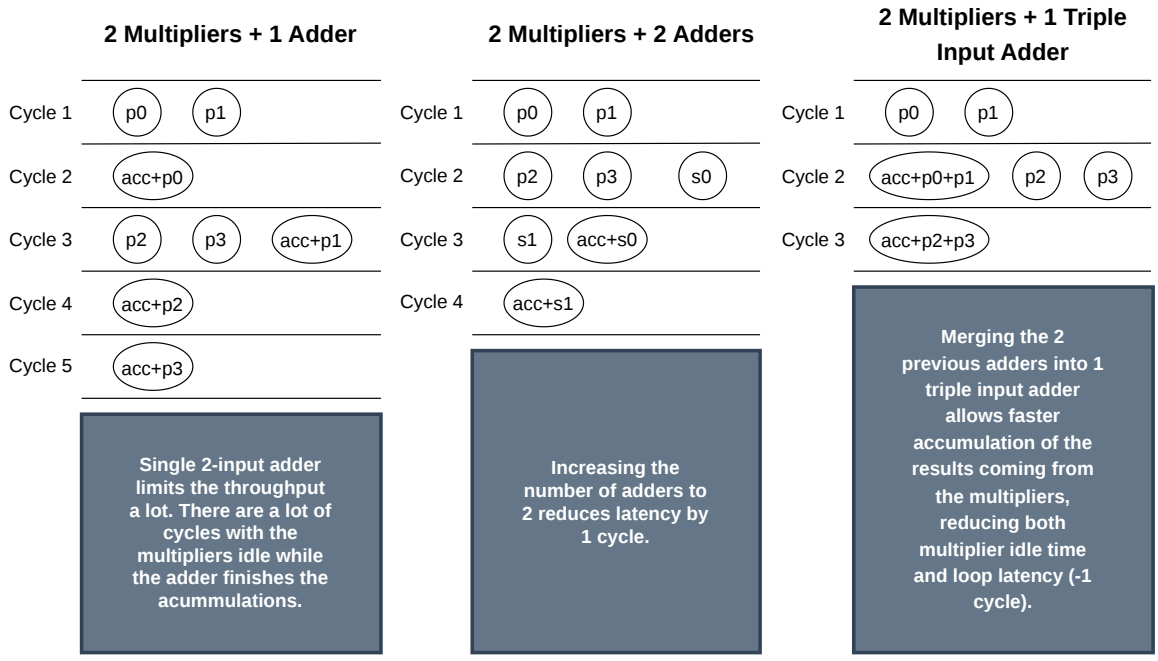


Figure 3: Different scheduling approaches.

3 Circuit design

3.1 Overall architecture

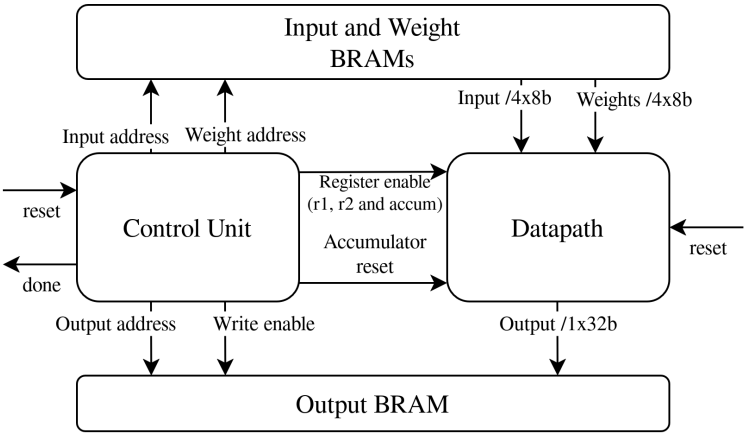


Figure 4: Overview of the system architecture.

3.2 Control Unit

3.2.1 Inputs and outputs

The control unit operates independently. Unlike the first project, it takes no inputs from the interface as there isn't one. Its only input is the reset signal. From reset onwards, it is solely responsible for everything that happens in the circuit.

The control unit provides the datapath with 5 binary signals: 3 register enables and two selector controls. It also provides the circuit with signals to control the BRAMs, namely, three address buses and a write enable for the output memory. Finally, it also outputs a `done` indicator to signal the completion of the processing.

3.2.2 State machine

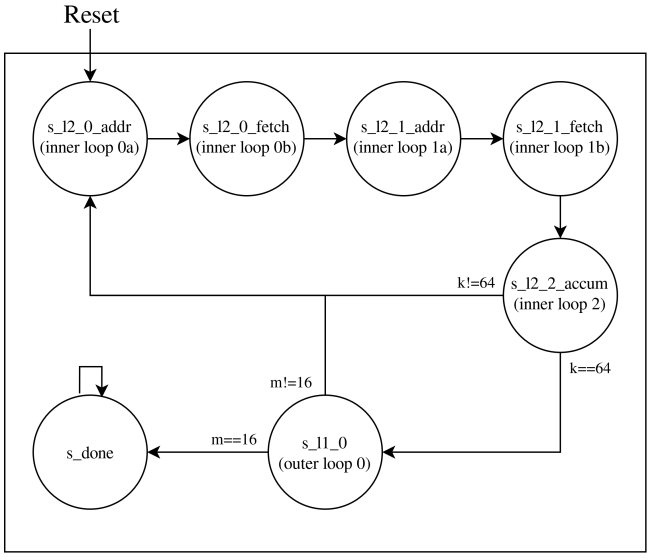


Figure 5: State machine diagram.

The state machine, represented in Figure 5, mimics the steps done in the example C provided to us, implemented in 7 states. There are two nested loops: inner and outer. In the state machine, these were renamed to l2 and l1 respectively for brevity.

The inner loop performs four multiply-accumulate operations per iteration, where two are performed in parallel by the two multipliers and the 3-input adder. Given that a total of 64 operations must be done per output, the inner loop does 64/4=16 iterations.

The outer loop iterates through the output neurons, in this case 16. In every iteration of the outer loop, the accumulator is reset, the 16 inner loop iterations are performed and the result is saved in the output memory.

3.3 Datapath

The datapath was built based on the chosen implementation. Since the design was limited to 2 multipliers, input multiplexers and an intermediate register are present, allowing operator sharing. The datapath architecture is shown in Figure 6, it contains 4 multiplexers and 3 registers.

The register and operator bit-width have been adjusted to optimize the design. All input values and weights are 8 bits wide. The width of the product between two values never exceeds the sum of the widths of each value. Therefore, only 16 bits are needed at the output of the multipliers. Given that there are 64 inputs, there will be 64 16-bit products to be added together. If we express the total sum of the products as a tree of two-input additions, each level in the tree introduces one extra bit of width. With 64 values, the tree will be $log2(64) = 6$ levels deep. Consequently, the width of the result will be $16 + 6 = 22$ bits. In our particular case, we use a three-input adder, which takes in the two 16-bit products and the 22-bit accumulator, so the adder itself must also be 22 bits wide.

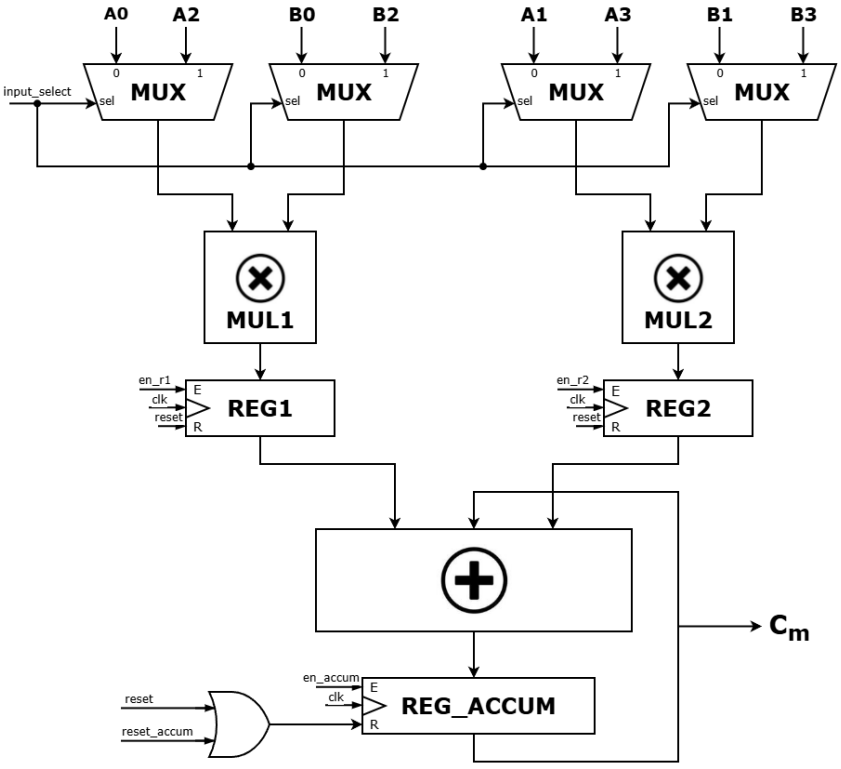


Figure 6: Diagram of the datapath unit showing all hardware blocks, interconnections, and signals involved.

3.4 Testbench

The testbench (`circuito_tb`) performs automated synchronous verification by comparing the circuit's output (`DataOut`) against predefined reference arrays (`EXPECTED_C`), utilizing assert statements to detect mismatches and log verification reports. Alongside nominal test cases (Mem 0 through 9) that reflect realistic operation

with partial zero outputs from ReLU activations, the testbench explicitly accounts for extreme cases by setting maximum negative inputs of -128, maximum positive inputs of $+127$, and purely negative accumulation patterns (-128×127) that verify whether the ReLU unit properly clips negative values to zero. Temporal synchronization is handled by sampling the data 15 clock cycles after the write-enable assertion (`we_d1`), allowing sufficient time for the output `BRAM` to present the result on `DataOut` and reliably validate the circuit's execution.

4 Synthesis results

Table 1 summarizes the post-synthesis FPGA resource utilization for the complete system, highlighting a minimal logic footprint of approximately 1.33% of Slice LUTs and 0.20% of Slice Registers, alongside 43.40% of the available IOBs. Notably, synthesis did not use any dedicated DSP slices, the two 8-bit signed multipliers were entirely mapped into distributed slice logic, utilizing general LUTs and 35 dedicated carry-chain adders (`CARRY4`) to compute the arithmetic products and partial accumulations. Furthermore, the memory subsystem allocated 3 Block RAM tiles (`RAMB36E1`) to host the Input Memory (`IM`), Weight Memory (`WM`), and Output Memory (`OM`). The RTL optimization mapped the 7-state FSM in the `control_unit` using a One-Hot encoding scheme to minimize state decoding latency. For the target 10 ns (100 MHz) clock period, the post-synthesis timing analysis achieves a Worst Negative Slack (WNS) of 1.326 ns and a Worst Hold Slack (WHS) of 0.070 ns, confirming comfortable timing margins prior to implementation. The synthesis strategy was set as the default because none of the others improved the result.

Table 1: FPGA resource utilization post-synthesis.

Resource	Used	Available	Utilization (%)
Slice LUTs	276	20 800	1.33
Slice Registers (FFs)	82	41 600	0.20
DSP Slices	0	90	0.00
Block RAM Tiles	3	50	6.00
Bonded IOBs	46	106	43.40

5 Implementation results

Following implementation, the design achieves timing closure under the target 10 ns (100 MHz) clock period. Comparing the timing results with post-synthesis, the Worst Negative Slack (WNS) decreased from 1.326 ns to 0.512 ns due to the physical routing delays of the distributed arithmetic logic, while the Worst Hold Slack (WHS) improved from 0.070 ns to 0.113 ns. Although the WNS indicates a theoretical minimum clock period of 9.488 ns ($f_{\max} \approx 105.4$ MHz), it was not possible to further reduce the clock period in practice because the synchronous Block RAMs require sufficient time to make their outputs available.

Table 2 shows that physical optimizations during placement and routing reduced the Slice LUT utilization from 276 to 266 (1.28%), while increasing the Slice Registers from 82 to 88 (0.21%).

Operating at 10 ns (100 MHz), the complete computation takes 12.96 μs, achieving a throughput of:

$$\text{Throughput} = \frac{\text{Total MACs}}{\text{Total Time}} = \frac{16 \times 64}{1296 \times 10 \text{ ns}} = \frac{1024 \text{ MACs}}{12.96 \ \mu\text{s}} \approx 79.01 \text{ MMAC/s} \tag{1}$$

where MMAC/s stands for million multiply-accumulate operations per second.

Table 2: FPGA resource utilization post-implementation.

Resource	Used	Available	Utilization (%)
Slice LUTs	266	20 800	1.28
Slice Registers (FFs)	88	41 600	0.21
DSP Slices	0	90	0.00
Block RAM Tiles	3	50	6.00
Bonded IOBs	46	106	43.40

The implementation strategy was set as the default because none of the others improved the result.

5.1 Critical path

According to Vivado, the critical path (`Path 1`) runs from the memory (`RAMB36E1`), through multiplier 1 (`MUL2_OUT`), to register 1 (`REG1`), causing a large delay (9.449 `ns`) mainly due to the memory access time (2.454 `ns`), the rest of the circuit (the adder and the accumulator register) is not included because timing analysis evaluates the maximum path delay within a single clock cycle.

6 Pipelining

6.1 Current architecture pipelining

Looking at Figure 6, it is easy to see that the current architecture can be easily converted into a 2-stage pipeline. While `p2` and `p3` are being accumulated, `p0` and `p1` of the next iteration can already be calculated. Registers 1 and 2 store the intermediate results and allow for the inter-iteration pipelining without adding more hardware. Stage 1 of the pipeline performs the multiplications and the second stage performs the accumulations.

This architecture operates as described below.

1. Set the addresses needed for the first 4 multiplications.
2. Wait to obtain the values from memory.
3. Calculate `p0` and `p1`. Set the addresses for calculating the next iteration.
4. Calculate `p2` and `p3`, accumulate `p0` and `p1` using the adder.
5. Accumulate `p2` and `p3`, finishing one iteration of the inner loop. Simultaneously, calculate `p0'` and `p1'`. Set the address for the next four products.
6. Accumulate `p0'` and `p1'`, calculate `p2'` and `p3'`.
7. Accumulate `p2'` and `p3'`, finishing another inner loop iteration. Simultaneously, calculate `p0''` and `p1''`.

To finish the first inner loop iteration, the system takes 5 cycles, but once the pipeline is full, the latency goes down to 3 cycles. Once this output is ready, there needs to be an extra cycle to allow writing to the memory and resetting the accumulator. The pipeline then has to refill.

This algorithm continues until all outputs have been calculated. In total, it takes 5 cycles per output for the first iteration, 3 cycles in the remaining 15 iterations, and one cycle between outputs for the resetting (totalling 15 additional cycles). Since there are 16 outputs, the pipelined architecture takes (5+3*15)*16+15=815 cycles.

Compared with the non-pipelined architecture, this is a $1 - \frac{815}{1296} \times 100 \approx 37$ % improvement. It results in a throughput of $\frac{1024}{815 \times 10 \times 10^{-9}} = 125.6$ MMAC/s

6.2 Maximum Throughput

The circuit performance can be improved by pipelining the architecture. On every read from the Input Memory (`IM`) and Weight Memory (`WM`), the datapath receives 4 inputs and 4 weights (32-bit words). With only 2 multipliers, the circuit takes two clock cycles just to multiply a single memory read, creating a bottleneck. With the addition of two extra multipliers, and changing the 3-input adder to a 5-input adder, it is possible to maximize the performance.

It is possible to divide the pipeline into 2 main stages:

- **Stage 1 (Multiplication):** Four 8-bit signed multipliers compute the products `p0, p1, p2`, and `p3` in parallel storing the 16-bit results in four pipeline registers. While the multipliers are operating, `IM` and `WM` simultaneously receive the next address.
- **Stage 2 (Addition and Accumulation):** The 22-bit 5-input adder sums the four registered products (`p0, p1, p2, p3`) with the 22-bit accumulator (`acc`) in a single clock cycle.

The bottleneck of the circuit remains the memory, since both the multiplication and the addition/accumulation stages can be executed every single clock cycle without stalling the pipeline. Because the 32-bit synchronous BRAMs can only deliver 4 inputs and 4 weights per clock cycle, the system is strictly limited to 4 MAC operations per cycle. In total, this pipelined architecture requires 4 multipliers (8-bit signed), 1 five-input adder (equivalent to four 2-input adders), and 5 registers (four 16-bit pipeline registers and one 22-bit accumulator register), with no input multiplexers needed since operators are not shared.

Once the pipeline is full, it completes 4 MACs every clock cycle, with a clock period of `10 ns`, the throughput can reach `400 MMAC/s`. To compute the 16 required outputs, each neuron needs $64/4 = 16$ memory accesses, 1 wait cycle to fill the pipeline, and 1 cycle to empty the pipeline and write the result to the output memory, taking $16 + 2 = 18$ clock cycles per neuron. For all 16 outputs, this results in a total latency of $18 \times 16 = 288$ clock cycles (2.88 μs), yielding an effective throughput of:

$$\text{Throughput} = \frac{1024 \text{ MACs}}{288 \times 10 \text{ ns}} = \frac{1024 \text{ MACs}}{2.88 \ \mu\text{s}} \approx 355.6 \text{ MMAC/s} \tag{2}$$

Compared to the non pipelined 2 multiplier architecture, which achieved a throughput of 79.01 `MMAC/s`, the pipelined circuit delivers a 4.5× performance improvement, and when compared to pipelined 2 multipliers architecture which achieves a throughput of `125.6 MMAC/s`, the performance with 4 multipliers is 2.83× better.

7 Conclusion

In this project, a hardware accelerator for a Fully-Connected Neural Network layer with ReLU activation was successfully designed, simulated, and implemented on an FPGA. The main objective was to maximize performance, which drove the architectural choice of a datapath featuring two parallel 8-bit multipliers and a three-input 22-bit adder.

While the initial theoretical scheduling suggested a total execution time of 784 clock cycles, timing simulations revealed critical memory read latencies inherent to synchronous BRAMs. Consequently, the control unit's state machine was adapted to include necessary wait states, resulting in a reliable final execution time of 1296 cycles. This adjustment effectively highlights the practical gap between theoretical dataflow models and realistic hardware constraints.

The physical implementation results demonstrated highly efficient resource utilization. The design occupied only 1.28% of the available Slice LUTs and mapped the arithmetic operations entirely to distributed logic and carry-chains, without consuming any dedicated DSP slices. The circuit successfully met timing closure for the target 10 ns clock period (100 MHz), yielding a computational throughput of approximately 79.01 MMAC/s.

Finally, rigorous testbench verification confirmed the circuit's mathematical accuracy across both nominal scenarios and extreme corner cases, successfully validating the multiply-accumulate precision and the correct zero-clipping behavior of the ReLU activation function. The final system proved to be a robust and optimized hardware solution.